

SECURITY OPERATIONS CENTER — HOME LAB

Phase 3

Firewalled Network, DMZ Zones & NAT

When the firewall becomes the attacker — and the SOC has to prove otherwise

A full engineering walkthrough for teams rebuilding or extending the lab

Detailed edition — firewall internals, every rule, NAT, the log-pipeline story, data flow and placement.

What Phase 3 sets out to do

- Replace the plain router with a real stateful firewall (OPNsense)
- Split the flat lab into THREE security zones + a DMZ
- Expose ONE real service to attack (a web server), hide everything else
- Introduce NAT — and watch it break source attribution on purpose
- Feed the firewall's own decisions into the SOC (blocks + NAT truth)
- Run a correlation exercise where no single alert tells the whole story

The through-line:

Phase 1 = can we see? · Phase 2 = can we see across a router? · Phase 3 = can we still name the attacker once a firewall lies?

What a firewall is (vs a router)

A router (Phase 2)

Job: move packets between subnets.

Asks: 'where does this go?'

Forwards EVERYTHING it can route.

No concept of allowed / denied.

No memory of past packets.

Keeps the real source address.

A firewall (Phase 3)

Job: decide AND move packets.

Asks: 'is this ALLOWED, then where?'

Forwards only what policy permits.

Every packet -> pass or block.

Stateful: tracks each connection.

Can rewrite the source (NAT).

OPNsense — the firewall we deployed

- OPNsense 26.7 — open-source firewall built on FreeBSD + pf
- One new VM replaces the Phase 2 VyOS router at the boundary
- Stateful packet filter (pf) + web GUI (webConfigurator) for policy
- Three interfaces, one per security zone (details next slide)
- Managed out-of-band via an SSH tunnel through a SOC jump host

GUI access (no direct route)

```
desktop
| ssh -L 8080:10.10.10.1:80
|      soc@192.168.56.20
v
wazuh (jump host)
| Intranet 10.10.10.20
v
OPNsense GUI 10.10.10.1:80
```

Three legs — one per zone

NIC	ROLE	ZONE	GATEWAY IP	VIRTUALBOX WIRE	HOLDS
em0	WAN	EXTRANET	10.10.0.1/24	intnet: extranet	the attacker's segment
em1	LAN	INTRANET	10.10.10.1/24	intnet: labnet	client + SOC (trusted)
em2	OPT1	DMZ	10.10.20.1/24	intnet: vlan20	web server + honeypot

Each firewall NIC is the default gateway (.1) for its subnet. The VirtualBox 'intnet' name is the virtual wire.

The full map — real addressing

EXTRANET (untrusted / lvl 0)

10.10.0.0/24

attacker .40

fw WAN em0 .1

INTRANET (trusted / lvl 100)

10.10.10.0/24

wazuh .20 zabbix .22

velo .23 grafana .24

client .50 fw LAN em1 .1

DMZ (exposed / lvl 50)

10.10.20.0/24

web server .60 nginx

honeypot .70 Cowrie

fw OPT1 em2 .1

SENSOR passive tap

Suricata + Zeek

MANAGEMENT (out-of-band)

192.168.56.0/24

every SOC box has a

2nd NIC here for admin

(never crosses zones)

Where everything sits, and why

Attacker	EXTRANET	outside the firewall — must come THROUGH it, like a real internet threat
-----------------	-----------------	--

Web server	DMZ	exposed on purpose; the one host the outside may reach
-------------------	------------	--

Honeypot	DMZ	bait on the exposed segment; catches lateral movement
-----------------	------------	---

Sensor	DMZ tap	where attacks LAND — a tap on the trusted side would see nothing
---------------	----------------	--

SOC stack	INTRANET	trusted zone; watches the DMZ from safety, through the firewall
------------------	-----------------	---

SOC admin	MANAGEMENT	out-of-band 192.168.56.x — control plane never rides the data path
------------------	-------------------	--

Placement is a security decision: the sensor watches where the danger is; management stays off the data path.

Why the sensor lives in the DMZ

A passive tap sees **ONLY** the segment it sits on.

If it were on the Intranet:

It would watch the client and SOC talk to each other — quiet, trusted traffic.
The attack on the web server happens in the DMZ. The tap would never see it.
'No alerts' would look like safety, while the DMZ is under active attack.

On the DMZ (our choice):

It sits exactly where the exposed hosts and the attacker's packets meet.
Every scan, SQLi and LFI against the web server is captured at the source.
Promiscuous mode + a mirror of the DMZ wire = full packet visibility.

Security zones — trust flows downhill

EXTRANET

lvl 0

untrusted — the attacker. Trust nothing that starts here.

DMZ

lvl 50

exposed services. Assume every host here is already breached.

INTRANET

lvl 100

client + SOC. The crown jewels. Protect at all costs.

Higher trust MAY initiate to lower. Lower trust may NEVER climb — except through audited holes.

Reading an OPNsense rule set

- Rules attach to the interface where traffic ENTERS the firewall
- So 'who may reach the DMZ' lives on the EXTRANET tab, not the DMZ tab
- Evaluated TOP-DOWN, first match wins — order is everything
- Default (except LAN) is DENY: no rule = the packet is dropped
- Stateful: allow the first packet, the return traffic is auto-permitted
- A 'block' rule with logging is how a silent drop becomes visible

Mental model

For traffic arriving on a zone's wire: walk that zone's rules top-down; the first match (pass or block) decides. No match = deny.

What the attacker's zone may do

1 PASS

```
tcp  EXTRANET:any  ->  10.10.20.60  : 80
```

publish ONLY the web server, HTTP only

2 BLOCK+log

```
any  EXTRANET  ->  INTRANET net
```

attacker can never touch client or SOC

3 BLOCK+log

```
any  EXTRANET  ->  This Firewall
```

attacker can never reach the firewall itself

Note: source PORT must be 'any' — only the DESTINATION port is 80. (This bit us; see the bugs slide.)

Result: from the outside, the entire lab looks like a single web server. Everything else is invisible.

One published service, two logged denials. The attacker's whole world is 10.10.20.60:80.

Containing the exposed zone

1 PASS

```
tcp  DMZ  ->  SOC_SERVERS ( .20/.22/.23 )
```

the monitoring hole: agents report to the SOC

2 PASS

```
tcp  web .60  ->  client .50 : 22
```

the ONE deliberate pivot path (SSH)

3 BLOCK+log

```
any  DMZ  ->  anything else
```

a compromised DMZ host is trapped here

SOC_SERVERS is a named alias {10.10.10.20, .22, .23} — one rule instead of three, and no hardcoding.

Rule 2 is the intentional weakness that becomes the attack's pivot path in the correlation exercise.

Two narrow allows (telemetry + one pivot), then deny-and-log everything. Breach containment, written down.

The trusted zone

1 PASS any INTRANET -> any

The trusted zone may initiate anywhere (it polls the DMZ, reaches admin, etc.).

- Inbound is still protected: default-deny on every OTHER zone stops
- the outside from ever starting a connection into the Intranet.
- Stateful returns mean the DMZ/Extranet can only REPLY, never initiate in.
- This asymmetry is the whole point: trusted reaches out, nothing reaches in.

server A — one real service, on purpose

- server-b was re-wired from a leftover segment onto the DMZ (10.10.20.60)
- Runs nginx serving a 'Nuetel Customer Operations Portal' (looks real)
- The ONLY thing the firewall lets the Extranet reach (port 80)
- Bait endpoints invite attack: search.php?q= , download.php?file= , admin.php
- Its nginx access.log becomes an independent 4th evidence source

:80

THE ONE DOOR

left open on purpose

What NAT is, from scratch

NAT = Network Address Translation. As a packet crosses the firewall, the firewall **REWRITES** an address in it and remembers the swap so replies can be un-swapped.

Source NAT (what we use)

```
before:  src=10.10.20.70  dst=10.10.0.40  (real host -> attacker)
after :   src=10.10.0.1   dst=10.10.0.40  (firewall's WAN IP!)
```

the firewall keeps a state entry so the reply is mapped back,
but ON THE WIRE the original source 10.10.20.70 is GONE.

How each phase treats the source

Phase 1 Flat

no boundary at all

src IP always real

Phase 2 Routed

VyOS routes, never rewrites

src IP always real

Phase 3 Firewallled

OPNsense Automatic Source NAT

src rewritten on egress

The jump: for the first time, a log can name the WRONG source. Attribution is no longer free.

Phase 2 deliberately used no-NAT so attribution stayed intact. Phase 3 takes that away to teach the recovery.

Two phases of honest addresses, then Phase 3 introduces the lie the SOC must learn to see through.

Automatic Source NAT is narrow

It rewrites ONLY traffic leaving the WAN (Extranet). Everything else keeps the real IP.

FLOW	DIRECTION	NAT?	SOURCE SEEN
attacker -> web :80	Extranet -> DMZ (inbound)	NO	real 10.10.0.40
web -> client (pivot)	DMZ -> Intranet (inter-zone)	NO	real 10.10.20.60
web/honeypot -> attacker	DMZ -> Extranet (OUTbound)	YES	10.10.0.1 (firewall!)

Only the C2 callback is masked. Inbound attacks and the pivot keep real IPs — so those planes stay honest.

How any alert reaches Grafana

```
ENDPOINT plane      NETWORK plane
Wazuh agent          sensor: Suricata eve.json + Zeek
|                    | (read by a Wazuh agent as JSON)
+-----+-----+
      v
Wazuh MANAGER (analysisd: decoders -> rules)
      | writes matched alerts ->
/var/ossec/logs/alerts/alerts.json
      v
Filebeat -> OpenSearch indexer -> Grafana
```

Every plane funnels into the manager, becomes a JSON alert, is shipped to the index, and drawn in Grafana.

Why we needed the firewall's logs

The sensor has a blind spot the firewall alone can fill.

- A BLOCKED attack never reaches the DMZ wire — so the sensor never sees it.
- Example: attacker -> Intranet is dropped at the boundary. Sensor: silence.
- Only the firewall knows a cross-zone attempt was even MADE and denied.
- And when NAT masks a source, only the firewall's state table holds the truth.

Two witnesses, two questions

Sensor answers 'was it malicious?' Firewall answers 'allowed or denied — and who was really behind it?'

The scenario that forced a rethink

Goal: get OPNsense's block/pass events into Wazuh so they land in Grafana beside the sensor and endpoint alerts.

- First approach: Wazuh's built-in receiver (wazuh-remoted, UDP 514).
 - OPNsense -> syslog -> remoted. Clean, one component, no extra software.
- We proved each piece worked in isolation:
 - packets arrived (tcpdump) · decoder parsed it (logtest) · rule 87701 fired
 - · analysisd healthy · alert threshold fine (level 3).
- Yet where we looked — alerts.log — NOTHING appeared. For over an hour.

The red herring

The real bug was US, not the pipeline

This Wazuh writes alerts to alerts.JSON only — NOT the classic alerts.log.
We were grepping the wrong file the entire time. The alerts existed all along.
Proof: rule 87702 ('multiple blocks, same source', MITRE T1110) sat in alerts.json, decoded by 'pf', naming src 10.10.20.70 — exactly as designed.

- Lesson 1: verify the OUTPUT location before doubting the whole pipeline.
- Lesson 2: alerts.json is the source of truth — it is what Filebeat ships to Grafana.

Why we kept rsyslog anyway

Mid-debug we switched from Wazuh's remoted to the OS rsyslog daemon. Even after we found the real bug, we kept rsyslog — it is the more robust, standard design.

- rsyslog is the industry-standard way to centralise network-device logs.
- It DECOUPLES ingestion from Wazuh: syslog lands in a file we own and can filter.
- We filter by source (only 10.10.10.1) into /var/log/opnsense.log — clean and auditable.
- Wazuh then reads that file as a normal <localfile> — the exact path logtest validated.
- It survives Wazuh restarts and quirks in remoted's syslog forwarding.

One catch we had to fix:

rsyslog's default template writes ISO timestamps; Wazuh's syslog predecoder needs BSD. Fix = `RSYSLOG_TraditionalFileFormat`.

How a block becomes a Grafana alert

```
OPNsense pf blocks a packet
  | filterlog writes a CSV event
  v syslog UDP 514
rsyslog on wazuh (if src==10.10.10.1)
  | append, BSD-timestamp template
  v
/var/log/opnsense.log
  | Wazuh <localfile> log_format syslog
  v
analysisd -> decoder 'pf' -> rule 87701 (single) / 87702 (flood, T1110)
  | writes -> alerts.json
  v
Filebeat -> OpenSearch -> Grafana 'Firewall Blocks' panel
```

From CSV line to a named threat

```
filterlog[..]: 67,,,<uuid>,em2,match,block,in,4,..,icmp,84,  
10.10.20.70,10.10.10.50,datalength=64
```

- Wazuh SHIPS the OPNsense/pfSense support — no custom decoder needed:
- decoder 0455-pfsense_decoders ('pf') + rules 0540-pfsense_rules
- Rule 87701 (level 5) = a single firewall block
- Rule 87702 (level 10) = many blocks, same source -> MITRE T1110 brute-force
- Attributed to agent 000 (the manager) — the firewall has no agent of its own

One attack, four witnesses

1 RECON + WEB ATTACK

attacker 10.10.0.40 -> web 10.10.20.60 SQLi / LFI / sqlmap
seen by: NETWORK (Suricata) + nginx access.log

2 PIVOT (through the one allowed hole)

web 10.10.20.60 -> client 10.10.10.50 : 22 SSH brute force
seen by: ENDPOINT (Wazuh rule 5712 on the client)

3 C2 CALLBACK

honeypot/web -> attacker : 4444 (NAT masks src as 10.10.0.1)
seen by: FIREWALL block + NAT state table

The attribution break

NETWORK PLANE (Suricata) — sees the truth

10.10.0.40 -> 10.10.20.60 LFI / etc-passwd / sqlmap

SRC = 10.10.0.40 (real)

ENDPOINT PLANE (Wazuh, client) — blames the pivot

rule 5712 sshd brute force -> throwaway

SRC = 10.10.20.60 (wrong!)

The client believes its OWN web server is attacking it. One plane, alone, lies.

How the analyst recovers the truth

1 Endpoint alarm

client brute-forced by 10.10.20.60 — our own web server?!

2 Question it

why would a DMZ web server attack the trusted inside?

3 Pivot to network

Suricata: 10.10.0.40 hit .60 with LFI+sqlmap moments before

4 Confirm on firewall

10.10.0.40 -> Intranet was BLOCKED — it was forced to pivot

5 Verdict

true origin = 10.10.0.40; the web server is a stepping stone

Trace the incident across the SOC

```
web attack    -> DMZ tap -> Suricata eve.json -> Wazuh agent -> manager  
              -> alerts.json (9000010, sqlmap) -> index -> Grafana NETWORK
```

```
pivot SSH     -> client auth.log -> Wazuh agent -> manager  
              -> alerts.json (rule 5712) -> index -> Grafana ENDPOINT
```

```
direct try    -> firewall blocks -> filterlog -> rsyslog -> localfile  
              -> analysisd (87701/2) -> alerts.json -> Grafana FIREWALL
```

```
one timeline in Grafana, three planes, correlated by IP + time
```

Verified — through the firewall

●	Segmentation	attacker -> Intranet / firewall	BLOCKED + logged
●	Published svc	attacker -> web :80	ALLOWED (only this)
●	Monitoring hole	DMZ agents -> SOC_SERVERS	telemetry flows
●	Network plane	Suricata on the DMZ tap	real 10.10.0.40 named
●	Endpoint plane	Wazuh rule 5712 on client	pivot detected
●	Firewall plane	filterlog -> Grafana panel	87701 / 87702 shown

Every boundary tested with the firewall's own Live View — pass/block confirmed before anything was recorded.

The bugs that cost us hours

WAN publish rule dead

Source Port was set to HTTP — clients use random src ports. Set src port = any.

Firewall alerts 'missing'

They land in alerts.json, NOT alerts.log (JSON-only). Grep the JSON.

localfile duplicated

ossec.conf has many `</ossec_config>`; a sed-insert duplicates it. Append a block.

Wazuh ignored the logs

rsyslog wrote ISO timestamps; predecoder needs BSD. `RSYSLOG_TraditionalFileFormat`.

Snapshot deadlock

live-snapshot of the DB VM (zabbix) hangs. Always cold-snap: power off first.

Alias resolved empty

OPNsense aliases need their OWN Apply, separate from the rules Apply.

Rebuild & extend

Rebuild from scratch

- Restore snapshot phase3-topology-complete
- GUI: ssh -L 8080:10.10.10.1:80
- soc@192.168.56.20
- Rules: Firewall > Rules, per zone
- Logs: /etc/rsyslog.d/30-opnsense.conf

Ways to improve it

- Grafana firewall-overview row
- Wazuh agent ON the web server
- Real vuln app (DVWA) behind nginx
- Suricata IPS mode (active block)
- Tune 87702 threshold per zone

Golden rules for editing this lab

- One published service per DMZ host — least privilege, always logged.
- Never a second default route — route only the lab subnets via the firewall.
- Cold-snapshot before every change; the DB VM will not live-snapshot.
- Grep alerts.json, not alerts.log. Aliases and rules each need their own Apply.

Phase 3 — PASSED

- OPNsense firewall — three enforced security zones + a DMZ
- One published web server; everything else denied and logged
- NAT rewrites egress — attribution breaks by design
- Firewall logs flow to Wazuh -> Grafana (87701 / 87702)
- Correlation exercise unmask a pivot the endpoint misblamed
- All persisted; snapshot: phase3-topology-complete

Next — Phase 4

Two firewalls + a site-to-site VPN. Encryption blinds the sensor; detection shifts to endpoints and flow metadata across a WAN boundary. Flat > Routed > [Firewalled+NAT] > Two firewalls + VPN